Lab 9A

Activity #1

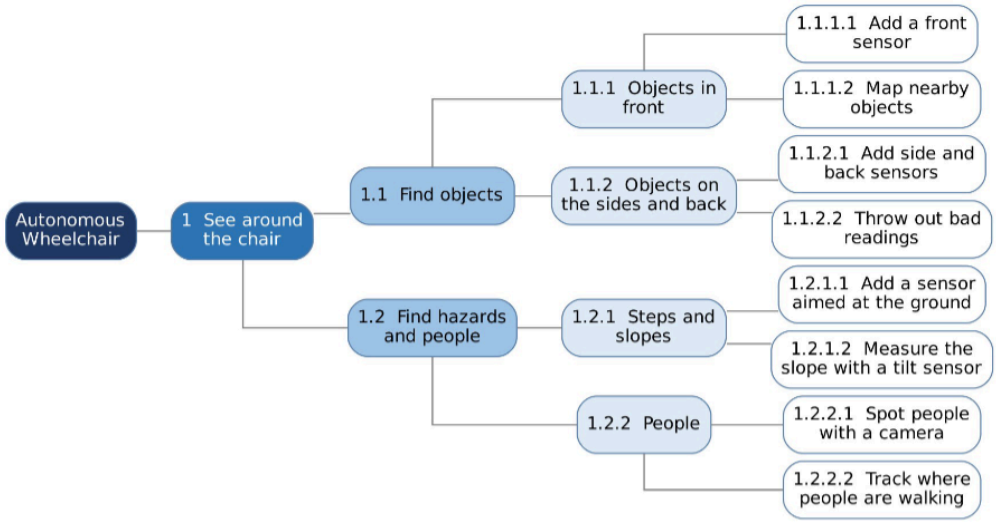
Engineering Design Problem:

Topic:

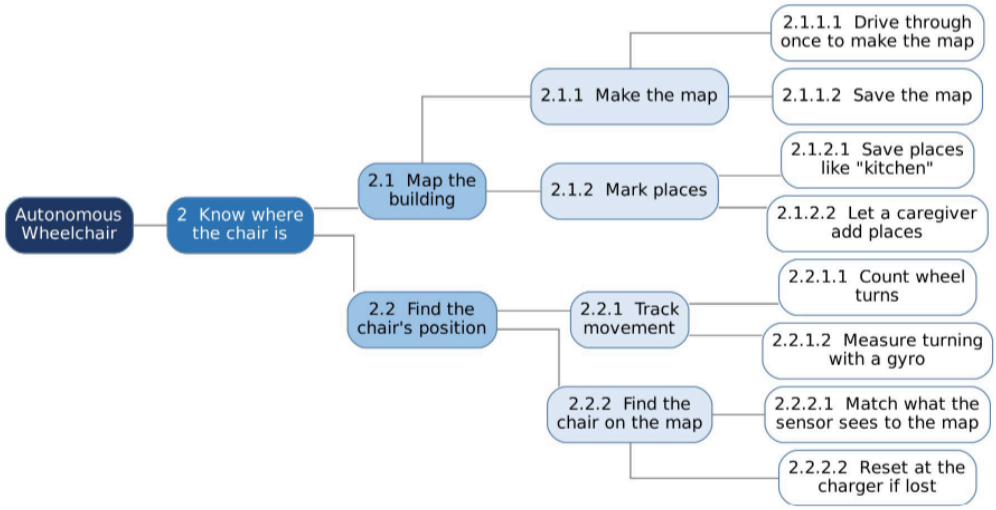
An autonomous personal movement device – essentially an autonomous wheelchair. The device should operate with the same capabilities as a standard motorized wheelchair, but be able to operate autonomously, rather than being controlled directly by the person in the chair.

Hierarchy:

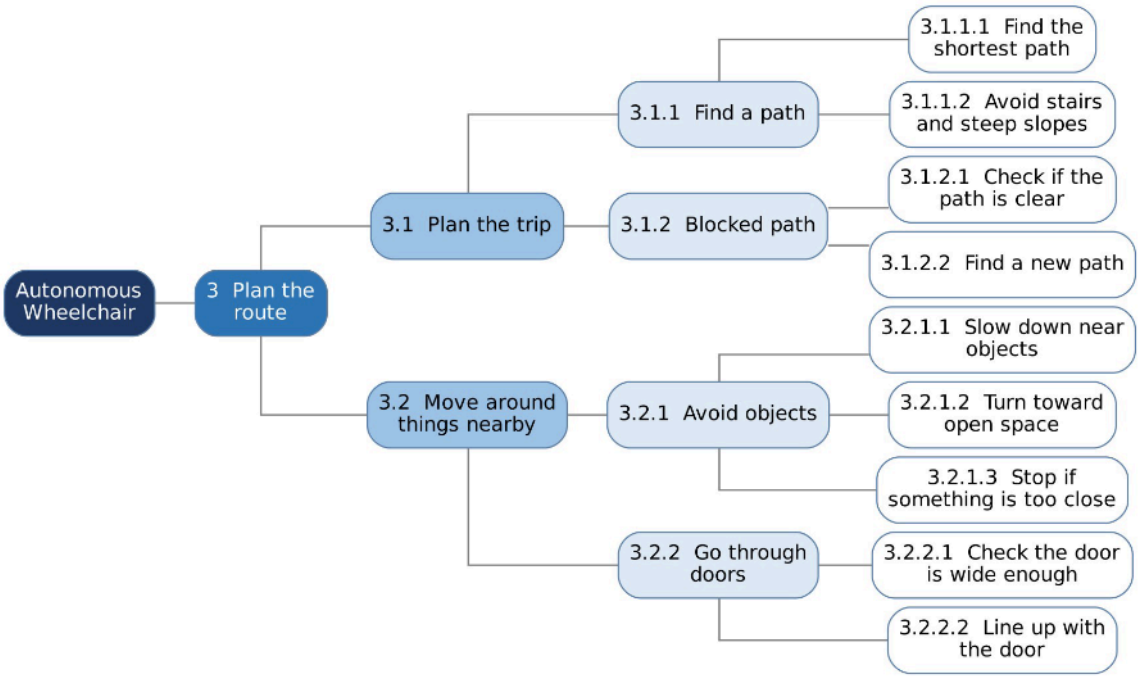
Branch 1: See around the chair



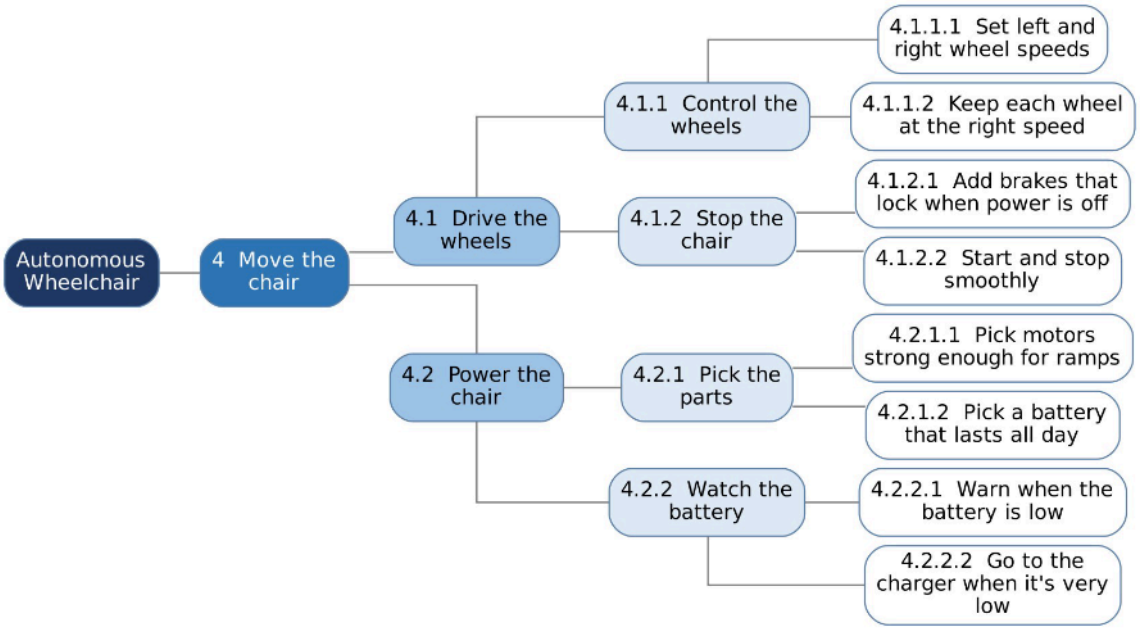
Branch 2: Know where the chair is



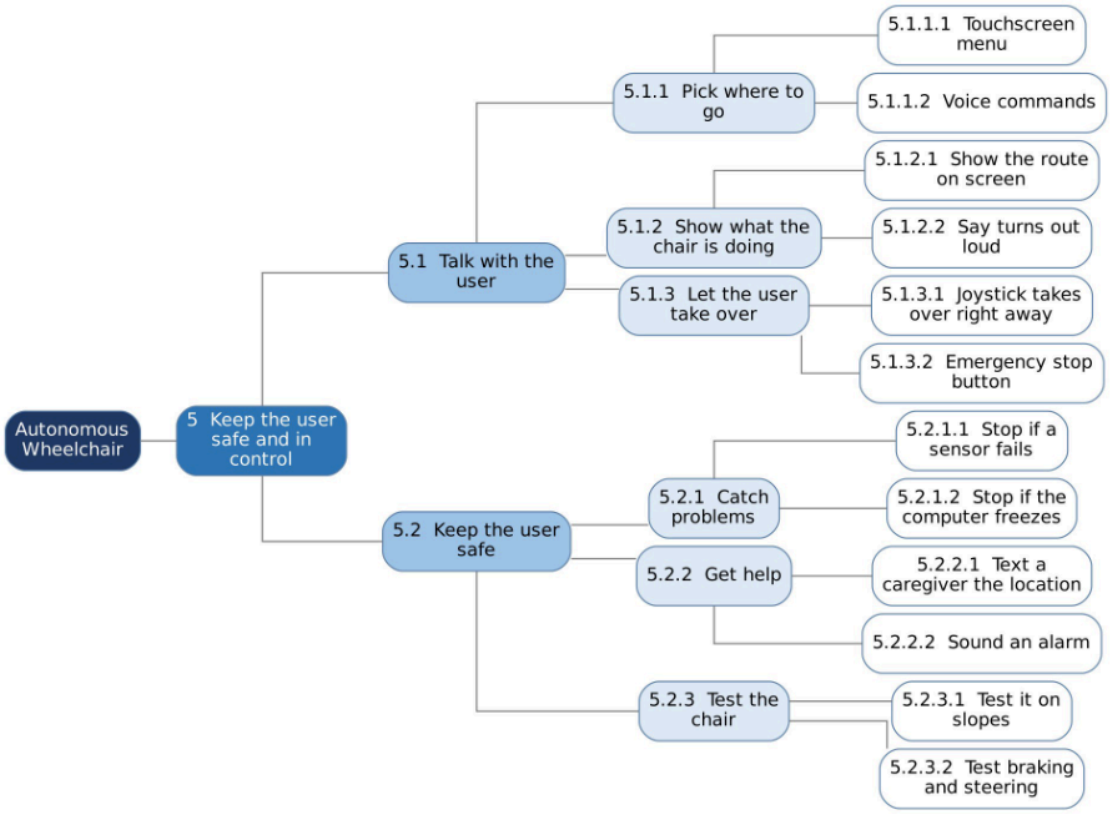
Branch 3: Plan the route



Branch 4: Move the chair



Branch 5: Keep the user safe and in control



Activity #2:

Top Down Design of a Program:

Hierarchy:

1. Get Data Points
   1.1 Ask how many points
      1.1.1 Repeat until number is greater than 1
   1.2 Get x and y values
      1.2.1 Ask for x and y for each point
   1.3 Store data points
      1.3.1 Add [x, y] to points list
2. Get Target X
   2.1 Ask for x-value
   2.2 Validate input
      2.2.1 If not a number, ask again
3. Calculate Estimated Y
   3.1 Find nearest points
      3.1.1 Sort data points
   3.2 Choose interpolation or extrapolation

3.2.1 Target outside the data: extrapolation

3.2.2 Target inside the data: interpolation

3.3 Calculate y-value

3.3.1 Find slope

3.3.2 Find estimated y

4. Display Result

4.1 Print y-value

4.2 Print a descriptive sentence

Variables: (With 1 sentence explaining what each variable does)
points - gets the list of [x, y] pairs entered by the user
n - gets an input from the user of how many points they are going to enter
data - the user's points after they are sorted by x value
target_x - gets an input from the user of the x value to find a y value at
point1 / point2 - the two data points used for the estimate (the nearest or surrounding points)
x1, y1, x2, y2 - the x and y values pulled out of point1 and point2
slope - calculated via the equation (y2-y1)/(x2-x1)
solution - the estimated y value at target_x, calculated as y1 + slope*(target_x - x1)
method - stores whether the estimate was "Interpolation" or "Extrapolation"

Test Cases:

Test Case 1: Interpolation Inputs:
Number of points: 3 Points: (1, 2), (3, 6), (5, 10) Target x: 2
Expected Output: The estimated y value at x = 2.0 is 4.0.

Test Case 2: Extrapolation (Below Range)
Inputs:Number of points: 3 Points: (2, 4), (4, 8), (6, 12) Target x: 0
Expected Output: The estimated y value at x = 0.0 is 0.0.

Edge Cases:

get_data_points() Edge Cases:
If x <= 1, reask the user for input until n is greater than or equal to 2

get_x_value() Edge Cases:
-Entering valid integers instead of decimals (should convert to a float successfully).
-Entering negative numbers or zero
-Typing letters or symbols for the target x value (the try-except block should catch this, print "Invalid Output!", and ask again)

Short Summary:

A. Combining our separately implemented modules/code was pretty challenging because each part was written by us separately. Further, when we combined the code, we immediately ran into mismatched function calls and conflicting logic. This phase showed us that we should have defined functions and data types before dividing up the work.

B. Dividing the program into sections provided more drawbacks than benefits in our opinion. The reason for this is because it made the development process feel way longer. But, one benefit I can see is with a very very long assignment/code writing I can see this being useful for organization. Lastly, like I said for a short program this felt more time consuming then necessary.

Code:

```
def get_data_points():
# 1. Get Data Points
    points = []

# 1.1 Ask how many points
    running = True
    while running:
        n = int(input("How many points?: "))

        if n <= 1:
            print("Only print integer numbers greater than 1")
        else:
            running = False

# 1.2 Get x and y values

    for i in range(n):
        x = float(input("Enter x value: "))
        y = float(input("Enter y value: "))
        # 1.3 Store data points
        points.append([x,y])

    return points;


def get_x_value():

    # 2. Get Target X
    #2.1 Ask for x-value
    running = True
    while running:
```

```
        try:
            x_value = float(input("What is the target x value?: "))
            running = False
        except ValueError:
            print("Invalid Output!")
    return x_value
    #2.2 Validate input



def calculate():
 # 3. Calculate Estimated Y
    # 3.1 Find nearest points
    data = get_data_points()
    data.sort()
    target_x = get_x_value()

    # 3.2 Choose interpolation or extrapolation
    for i in range(len(data) - 1):

        if target_x < data[0][0]:
            point1 = data[0]
            point2 = data[1]
            method = "Extrapolation"
            break

        elif target_x > data[-1][0]:
            point1 = data[-2]
            point2 = data[-1]
            method = "Extrapolation"
            break

        elif data[i][0] <= target_x <= data[i+1][0]:
            point1 = data[i]
            point2 = data[i+1]
            method = "Interpolation"
            break

    # 3.3 Calculate y-value


    x1 = point1[0]
    x2 = point2[0]
    y1 = point1[1]
```

```
    y2 = point2[1]

    # calculate slope
    slope = (y2-y1) / (x2-x1)
    # calculate estimated y value
    solution = y1 + slope * (target_x - x1)

    return solution, method, target_x


def display(solution, method, target_x):
    # 4. Display Result
    # 4.1 Print y-value
    # 4.2 Print a descriptive sentence
    print(f"The estimated y value at x = {target_x} is {solution}. This was calculated using
{method}.")



solution, method, target_x = calculate()
display(solution, method, target_x)
```